

Data Structure lab programs

Program 1: Implement basic array operations: insertion, deletion, searching, and traversal.

```
// implement basic array operations:insertion.deltion.searchingand traversal
#include<stdio.h>
int main()
{
    int arr[100],n,i,pos,value,choice,found;
    printf("enter the number of elements:");
    scanf("%d",&n);
    printf("enter elements\n");
    for(i=0;i<5;i++){
        scanf("%d",&arr[i]);
    }
    do{
        printf("----MENU----\n");
        printf("1.traversal\n 2.insertion \n 3.deletion \n 4.searching\n 5.exit\n");
        printf("enter your choice");
        scanf("%d",&choice);
        switch(choice){
            case 1://traversal
                printf("array elements are:");
                for(i=0;i<5;i++){
                    printf("%d",arr[i]);
                }

                break;
            case 2://insertion
                printf("enter position and value");
                scanf("%d%d",&pos,&value);
                for(i=0;i<5;i--){
                    arr[i] =arr[i-1] ;
                }
                arr[pos-1]=value;
                n++;
                break;
            case 3://deletion
                printf("enter position to delete");
                scanf("%d",&pos);
                for(i=pos-1;i<n-1;i++){
                    arr[i]=arr[i+1];
                }
                n--;
                break;
        }
    }
```

```

        case 4://searching
        printf("enter element to search");
        scanf("%d",&value);
        found=0;
        for(i=0;i<n;i++){
            if (arr[i]==value){
                printf("element found at position %d\n",i+1);
                found=1;
                break;
            }
        }
        if(found==0)
            printf("element not found\n");
        break;
        case 5:
        printf("exiting...\n");
        break;
        default:
        printf("invalid choice\n");
    }

    while(choice!=5);
    return 0;
}

```

Output:

```

enter the number of elements:5
enter elements
10 20 30 40 50
----MENU----
1.traversal
2.insertion
3.deletion
4.searching
5.exit
enter your choice1
array elements are:1020304050----MENU----
1.traversal
2.insertion
3.deletion
4.searching
5.exit
enter your choice|

```

Program 2: Represent a sparse matrix using triplet form and perform transpose operation.

```
//Represent a sparse matrix using triplet form and perform transpose operation
#include<stdio.h>

int main()
{
    int matrix[10][10], triplet[50][3], transpose[50][3];
    int i, j, m, n, k=1;

    printf("Enter rows and columns :");
    scanf("%d %d", &m, &n);

    printf("Enter matrix elements : \n");
    for(i=0; i<m; i++) {
        for(j=0; j<n; j++) {
            scanf("%d", &matrix[i][j]);
        }
    }

    //convert to triplet form
    triplet[0][0]=m;
    triplet[0][1]=n;
    triplet[0][2]=0;

    for(i=0; i<m; i++) {
        for(j=0; j<n; j++) {
            if(matrix[i][j]!=0) {
                triplet[k][0]=i;
                triplet[k][1]=j;
                triplet[k][2]=matrix[i][j];
                k++;
                triplet[0][2]++;
            }
        }
    }

    printf("\n Triplet Representation : \n");
    for(i=0; i<k; i++) {
        printf("%d %d %d \n", triplet[i][0], triplet[i][1], triplet[i][2]);
    }

    //Transpose
    transpose[0][0]=triplet[0][1];
    transpose[0][1]=triplet[0][0];
    transpose[0][2]=triplet[0][2];

    k=1;

    for(i=0; i<n; i++) {
        for(j=1; j<=triplet[0][2]; j++) {
            if(triplet[j][1]==i) {
                transpose[k][0]=triplet[j][1];
                transpose[k][1]=triplet[j][0];
                transpose[k][2]=triplet[j][2];
                k++;
            }
        }
    }

    printf("\n Transpose Triplet: \n");
    for(i=0; i<k; i++) {
        printf("%d %d %d \n", transpose[i][0], transpose[i][1], transpose[i][2]);
    }

    return 0;
}
```

Output:

```
Enter rows and columns :2 2
Enter matrix elements :
1 2 3 4

Triplet Representation :
2 2 4
0 0 1
0 1 2
1 0 3
1 1 4

Transpose Triplet:
2 2 4
0 0 1
0 1 3
1 0 2
1 1 4

Process returned 0 (0x0)   execution time : 9.125 s
Press any key to continue.
|
```

Program 3: Implement stack operations using arrays and linked lists.

Program 3a: Implement stack operations using arrays.

```
//implement stack operations using arrays and linked lists using array
#include<stdio.h>
#define MAX 5

int stack[MAX];
int top=-1;

//Push
void push(int value){
    if(top==MAX-1){
        printf("Stack Overflow\n");
    }else{
        top++;
        stack[top]=value;
        printf("Pushed %d\n",value);
    }
}

//Pop
void pop(){
    if (top== -1)
        printf("Stack underflow");
    else{
        printf("Popped %d\n",stack[top]);
        top--;
    }
}

//Display
void display(){
    if(top== -1)
        printf("Stack is empty\n");
    else{
        printf("Stack elements:\n");
        for(int i=top;i>=0;i--){
            printf("%d\n",stack[i]);
        }
    }
}

//Main
int main(){
    push(10);
    push(20);
    push(30);
    push(40);
    pop(10);
    display();
}
```

Output:

```
Pushed 10
Pushed 20
Pushed 30
Pushed 40
Popped 40
Stack elements:
30
20
10

Process returned 0 (0x0)   execution time : 0.026 s
Press any key to continue.
|
```

Program 3b: Implement stack operations using linked lists.

```
//stack implementation using linked list
#include<stdio.h>
#include<stdlib.h>

//Define node structure
struct Node{
    int data;
    struct Node*next;
};
struct Node*top=NULL; //Top of stack

//Push operation
void push(int value){
    struct Node*newNode=(struct Node*)malloc(sizeof(struct Node));

    if(newNode==NULL){
        printf("Stack Overflow(Memory not available)\n");
        return;
    }
    newNode->data=value;
    newNode->next=top;
    top=newNode;
    printf("%d pushed to stack\n,value");
}

//Pop operation
void pop(){
    if(top==NULL){
        printf("Stack Underflow(Empty stack)\n");
        return;
    }

    struct Node*temp=top;
    printf("%d popped from stack \n",top->data);
    top=top->next;
    free(temp);
}

// Peek operation
void peek() {
    if (top == NULL) {
        printf("Stack is empty\n");
    } else {
        printf("Top element is: %d\n", top->data);
    }
}

// Traverse operation
void display() {
    if (top == NULL) {
        printf("Stack is empty\n");
        return;
    }
    struct Node* temp = top;
    printf("Stack elements: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display (Traverse)\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```

----- \ - - , ----- , ,

switch (choice) {
    case 1:
        printf("Enter value to push: ");
        scanf("%d", &value);
        push(value);
        break;
    case 2:
        pop();
        break;
    case 3:
        peek();
        break;
    case 4:
        display();
        break;
    case 5:
        printf("Exiting...\n");
        exit(0);
    default:
        printf("Invalid choice! Please try again.\n");
}
}
return 0;
}

```

Output:

```

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display (Traverse)
5. Exit
Enter your choice: 2
Stack Underflow(Empty stack)

```

Program 4: Convert an infix expression to postfix using a stack.

```
// Convert an infix expression to postfix using a stack
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char c){
    if(top >= MAX - 1){
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = c;
}

char pop(){
    if(top < 0){
        return '\0';
    }
    return stack[top--];
}

char peek(){
    if(top < 0){
        return '\0';
    }
    return stack[top];
}

int precedence(char op){
    switch(op){
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
        default:
            return 0;
    }
}
```



```

int isOperator(char c){
    return (c=='+' || c=='-' || c=='*' || c=='/' || c=='^');
}

void infixToPostfix(char *infix, char *postfix){
    int i = 0, j = 0;
    char c;

    while((c = infix[i++]) != '\0'){
        // If operand, add to output
        if(isalnum(c)){
            postfix[j++] = c;
        }
        // If '(', push to stack
        else if(c == '('){
            push(c);
        }
        // If ')', pop until '('
        else if(c == ')'){
            while(top >= 0 && peek() != '('){
                postfix[j++] = pop();
            }
            pop(); // remove '('
        }
        // If operator
        else if(isOperator(c)){ // FIXED typo
            if(c == '^'){ // right associative
                while(top >= 0 && precedence(peek()) > precedence(c)){
                    postfix[j++] = pop();
                }
            } else {
                while(top >= 0 && precedence(peek()) >= precedence(c)){ // FIXED typo
                    postfix[j++] = pop();
                }
            }
            push(c);
        }
    }

    // Pop remaining operators
    while(top >= 0){
        postfix[j++] = pop();
    }

    postfix[j] = '\0'; // IMPORTANT
}

int main(){
    char infix[MAX], postfix[MAX];

    printf("Enter an infix expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("Postfix Expression: %s\n", postfix);
    return 0;
}

```

Output:

```

Enter an infix expression: (a+b*c)//d+e*f
Postfix Expression: abc*+/d/ef*+

Process returned 0 (0x0)   execution time : 21.267 s
Press any key to continue.

```

Program 5: Evaluate a postfix expression using a stack.

```
//Evaluate a postfix expression using a stack.
#include <stdio.h>
#include <ctype.h>
#define MAX 100

int stack[MAX];
int top = -1;

// Push operation
void push(int value)
{
    top++;
    stack[top] = value;
}

// Pop operation
int pop()
{
    return stack[top--];
}

int main()
{
    char postfix[MAX];
    int i, op1, op2, result;

    printf("Enter postfix expression: ");
    scanf("%s", postfix);

    for(i = 0; postfix[i] != '\0'; i++)
    {
        // If character is operand
        if(isdigit(postfix[i]))
        {
            push(postfix[i] - '0');
        }
        // If character is operator
        else
        {
            op2 = pop();
            op1 = pop();

            switch(postfix[i])
            {
                case '+':
                    result = op1 + op2;
                    break;

                case '-':
                    result = op1 - op2;
                    break;

                case '*':
                    result = op1 * op2;
                    break;

                case '/':
                    result = op1 / op2;
                    break;

                case '%':
                    result = op1 % op2;
                    break;

                default:
                    printf("Invalid operator");
                    return 1;
            }

            push(result);
        }
    }

    printf("Result = %d", pop());

    return 0;
}
```

Output:

```
Enter postfix expression: 52+3*
Result = 21
Process returned 0 (0x0)   execution time : 5.510 s
Press any key to continue.
```

Program 6: Implement queue operations using arrays and linked lists.

Program 6a: queue operation using arrays

```
// program 6a:Queue operations using arrays
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

// Insert element
void enqueue(int value)
{
    if(rear == MAX - 1)
    {
        printf("Queue Overflow\n");
    }
    else
    {
        if(front == -1)
            front = 0;

        rear++;
        queue[rear] = value;

        printf("%d inserted\n", value);
    }
}

// Delete element
void dequeue()
{
    if(front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        printf("%d deleted\n", queue[front]);
        front++;
    }
}
```

```

// Display queue
void display()
{
    int i;

    if(front == -1 || front > rear)
    {
        printf("Queue is Empty\n");
    }--
    else
    {
        printf("Queue Elements:\n");

        for(i = front; i <= rear; i++)
        {
            printf("%d ", queue[i]);
        }

        printf("\n");
    }
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    dequeue();

    display();

    return 0;
}

```

Output:

```

10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30

Process returned 0 (0x0)   execution time : 0.028 s
Press any key to continue.
|

```

Program 6b: queue operations using linked list

```
// program 6b:Queue operations using Linked List
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node
{
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

// Insert element
void enqueue(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = NULL;

    if(rear == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }

    printf("%d inserted\n", value);
}
```

```

// Delete element
void dequeue()
{
    struct Node *temp;

    if(front == NULL)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        temp = front;

        printf("%d deleted\n", front->data);

        front = front->next;

        if(front == NULL)
            rear = NULL;

        free(temp);
    }
}

// Display queue
void display()
{
    struct Node *temp;

    if(front == NULL)
    {
        printf("Queue is Empty\n");
    }
    else
    {
        temp = front;

        printf("Queue Elements:\n");

        while(temp != NULL)
        {
            printf("%d ", temp->data);
            temp = temp->next;
        }

        printf("\n");
    }
}

```

```

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    dequeue();

    display();

    return 0;
}

```

Output:

```
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30

Process returned 0 (0x0)   execution time : 0.023 s
Press any key to continue.
```

Program 07: implement a circular queue using arrays.

```
//7.Implement a circular queue using arrays.
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node
{
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

// Insert element
void enqueue(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = NULL;

    if(rear == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }

    printf("%d inserted\n", value);
}
```

```

// Delete element
void dequeue()
{
    struct Node *temp;

    if(front == NULL)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        temp = front;

        printf("%d deleted\n", front->data);

        front = front->next;

        if(front == NULL)
            rear = NULL;

        free(temp);
    }
}

```

```

// Display queue
void display()
{
    struct Node *temp;

    if(front == NULL)
    {
        printf("Queue is Empty\n");
    }
    else
    {
        temp = front;

        printf("Queue Elements:\n");

        while(temp != NULL)
        {
            printf("%d ", temp->data);

```

```

            temp = temp->next;
        }
    }

    printf("\n");
}

```

```

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    dequeue();

    display();

    return 0;
}

```


Output:

```
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30

Process returned 0 (0x0)   execution time : 0.113 s
Press any key to continue.
```

Program 08: implement a priority queue using arrays or linked list

```
//8. Implement a priority queue using arrays or linked lists.
#include <stdio.h>
#define MAX 5
int pq[MAX];
int size = 0;
// Insert element in sorted order
void enqueue(int value)
{
    int i;
    if(size == MAX)
    {
        printf("Priority Queue Overflow\n");
        return;
    }
    // Insert based on priority (smaller number = higher priority)
    for(i = size - 1; i >= 0 && pq[i] > value; i--)
    {
        pq[i + 1] = pq[i];
    }
    pq[i + 1] = value;
    size++;
    printf("%d inserted\n", value);
}

// Delete highest priority element
void dequeue()
{
    int i;
    if(size == 0)
    {
        printf("Priority Queue Underflow\n");
        return;
    }
    printf("%d deleted\n", pq[0]);
    for(i = 0; i < size - 1; i++)
    {
        pq[i] = pq[i + 1];
    }
}
```

```

        size--;
    }
    // Display elements
    void display()
    {
        int i;
        if(size == 0)
        {
            printf("Priority Queue is Empty\n");
        }
        else
        {
            printf("Priority Queue Elements:\n");

            for(i = 0; i < size; i++)
            {
                printf("%d ", pq[i]);
            }
            printf("\n");
        }
    }

    int main()
    {
        enqueue(30);
        enqueue(10);
        enqueue(50);
        enqueue(20);

        display();

        dequeue();

        display();

        return 0;
    }

```

Output:

```

30 inserted
10 inserted
50 inserted
20 inserted
Priority Queue Elements:
10 20 30 50
10 deleted
Priority Queue Elements:
20 30 50

Process returned 0 (0x0)   execution time : 0.126 s
Press any key to continue.

```

Program 9: Implement singly linked list operations: insertion, deletion, and search.

```
//Implement singly linked list operations:Insertion,deletion, and search.
#include<stdio.h>
#include<stdlib.h>

struct Node{
    int data;
    struct Node* next;
};

//
int search(struct Node*head,int target);

struct Node* createNode(int data){
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=data;
    newNode->next=NULL;
    return newNode;
}

void insertAtBeginning(struct Node**head,int data){
    struct Node* newNode= createNode(data);
    newNode->next=*head;
    *head=newNode;
    return;
}

void insertAtEnd(struct Node**head,int data){
    struct Node* newNode=createNode(data);
    if(*head==NULL){
        *head=newNode;
        return;
    }
    struct Node* temp = *head;
    while(temp->next!=NULL){
        temp=temp->next;
    }
    temp->next=newNode;
}

void insertAtPosition(struct Node**head,int data,int position){
    struct Node* newNode=createNode(data);
    if(position==0){
        newNode->next=*head;
        *head=newNode;
        return;
    }
    struct Node* temp=*head;
    for(int i=0;temp!=NULL && i<position-1;i++){
        temp=temp->next;
    }
    if(temp==NULL){
        printf("Position out of range \n");
        return;
    }
    newNode->next=temp->next;
    temp->next=newNode;
}

void deleteFromBeginning(struct Node**head){
    if(*head==NULL)return;
    struct Node*temp=*head;
    *head=temp->next;
    free(temp);
}

void deleteFromEnd(struct Node**head){
    if(*head==NULL)return;
    if((*head)->next==NULL){
        free(*head);
        *head=NULL;
        return;
    }
}
```

```

    struct Node* temp=*head;
    while(temp->next->next !=NULL) {
        temp=temp->next;
    }

    free(temp->next);
    temp->next=NULL;
}

void deleteFromPosition(struct Node**head,int position){
    if(*head==NULL) return;

    struct Node*temp=*head;

    if(position==0){
        *head=temp->next;
        free(temp);
        return;
    }

    for(int i=0;temp!=NULL && i<position-1;i++){
        temp=temp->next;
    }

    if(temp==NULL||temp->next==NULL){
        printf("Position out of range\n");
        return;
    }

    struct Node* next=temp->next->next;
    free(temp->next);
    temp->next=next;
}

void traverse(struct Node*head){
    struct Node*temp=head;
    while(temp!=NULL){
        printf("%d->",temp->data);
        temp=temp->next;
    }
}

void traverse(struct Node*head){
    struct Node*temp=head;
    while(temp!=NULL){
        printf("%d->",temp->data);
        temp=temp->next;
    }
    printf("NULL\n");
}

int search(struct Node*head,int target){
    struct Node*temp=head;
    while(temp!=NULL){
        if(temp->data==target) return 1;
        temp=temp->next;
    }
    return 0;
}

int main(){
    struct Node*head=NULL;

    insertAtEnd(&head,10);
    insertAtEnd(&head,20);
    insertAtEnd(&head,30);
    traverse(head);

    insertAtBeginning(&head,5);
    traverse(head);

    insertAtPosition(&head,25,2);
    traverse(head);

    insertAtPosition(&head,25,2);
    traverse(head);

    deleteFromBeginning(&head);
    traverse(head);
}

```

```

deleteFromEnd(&head);
traverse(head);

deleteFromPosition(&head,1);
traverse(head);

printf("List contains 20:%s \n",search(head,20)?"Yes":"No");
printf("List contains 40:%s\n",search(head,40)?"Yes":"No");

return 0;
}

```

Output:

```

10->20->30->NULL
5->10->20->30->NULL
5->10->25->20->30->NULL
5->10->25->25->20->30->NULL
10->25->25->20->30->NULL
10->25->25->20->NULL
10->25->20->NULL
List contains 20:Yes
List contains 40:No

Process returned 0 (0x0)   execution time : 0.014 s
Press any key to continue.

```

Program 10: implement binary tree creation and traversal [preorder, inorder, postorder]

```
//10. Implement binary tree creation and traversal (preorder, inorder, postorder).
#include <stdio.h>
#include <stdlib.h>
// Structure for tree node
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
// Create new node
struct Node* createNode(int value)
{
    struct Node *newNode;
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
// Preorder Traversal
void preorder(struct Node *root)
{
    if(root != NULL)
    {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
// Inorder Traversal
void inorder(struct Node *root)
{
    if(root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
// Postorder Traversal
void postorder(struct Node *root)
{
    if(root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main()
{
    // Creating Binary Tree
    struct Node *root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);
    // Traversals
    printf("Preorder Traversal:\n");
    preorder(root);
    printf("\n\nInorder Traversal:\n");
    inorder(root);
    printf("\n\nPostorder Traversal:\n");
    postorder(root);
    return 0;
}
```

Output:

```
Preorder Traversal:
1 2 4 5 3

Inorder Traversal:
4 2 5 1 3

Postorder Traversal:
4 5 2 3 1
Process returned 0 (0x0)   execution time : 0.016 s
Press any key to continue.
```

Program 11: Represent a graph using adjacency matrix and adjacency list

Program 11a: using adjacency matrix

```
//11. Represent a graph using adjacency matrix and adjacency list.
//using adjacency matrix
#include <stdio.h>

#define MAX 10

int main()
{
    int graph[MAX][MAX];
    int vertices, edges;
    int i, j, src, dest;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    // Initialize matrix with 0
    for(i = 0; i < vertices; i++)
    {
        for(j = 0; j < vertices; j++)
        {
            graph[i][j] = 0;
        }
    }

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    // Input edges
    for(i = 0; i < edges; i++)
    {
        printf("Enter edge (source destination): ");
        scanf("%d %d", &src, &dest);

        graph[src][dest] = 1;
        graph[dest][src] = 1; // Undirected graph
    }
}
```



```

    }

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    // Input edges
    for(i = 0; i < edges; i++)
    {
        printf("Enter edge (source destination): ");
        scanf("%d %d", &src, &dest);

        graph[src][dest] = 1;
        graph[dest][src] = 1; // Undirected graph
    }

    // Display adjacency matrix
    printf("\nAdjacency Matrix:\n");

    for(i = 0; i < vertices; i++)
    {
        for(j = 0; j < vertices; j++)
        {
            printf("%d ", graph[i][j]);
        }

        printf("\n");
    }

    return 0;
}

```

Output:

```

Enter number of vertices: 3
Enter number of edges: 2
Enter edge (source destination): 0 1
Enter edge (source destination): 1 2

Adjacency Matrix:
0 1 0
1 0 1
0 1 0

Process returned 0 (0x0)   execution time : 18.262 s
Press any key to continue.
|

```

Program 11b: using adjacency list

```
//Using adjacency List

#include <stdio.h>
#include <stdlib.h>

#define MAX 10

// Node structure
struct Node
{
    int vertex;
    struct Node *next;
};

struct Node *adjList[MAX];

// Create new node
struct Node* createNode(int v)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->vertex = v;
    newNode->next = NULL;

    return newNode;
}

// Add edge
void addEdge(int src, int dest)
{
    struct Node *newNode;

    // Add edge src -> dest
    newNode = createNode(dest);
    newNode->next = adjList[src];

    adjList[src] = newNode;

    // Add edge dest -> src (Undirected graph)
    newNode = createNode(src);
    newNode->next = adjList[dest];
    adjList[dest] = newNode;
}

// Display adjacency list
void display(int vertices)
{
    int i;
    struct Node *temp;

    printf("\nAdjacency List:\n");

    for(i = 0; i < vertices; i++)
    {
        printf("%d -> ", i);

        temp = adjList[i];

        while(temp != NULL)
        {
            printf("%d -> ", temp->vertex);
            temp = temp->next;
        }

        printf("NULL\n");
    }
}

int main()
{
    int vertices, edges;
    int i, src, dest;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    // Initialize adjacency list
    for(i = 0; i < vertices; i++)
    {
        adjList[i] = NULL;
    }

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    // Input edges
    for(i = 0; i < edges; i++)
    {
        printf("Enter edge (source destination): ");
        scanf("%d %d", &src, &dest);

        addEdge(src, dest);
    }

    display(vertices);

    return 0;
}
```

Output:

```
Enter number of vertices: 3
Enter number of edges: 2
Enter edge (source destination): 0 1
Enter edge (source destination): 1 2

Adjacency List:
0 -> 1 -> NULL
1 -> 2 -> 0 -> NULL
2 -> 1 -> NULL

Process returned 0 (0x0)   execution time : 7.706 s
Press any key to continue.
|
```

Program 12: implement graph traversal : BFT and DFS

```
//12. Implement graph traversal: BFS and DFS.
#include <stdio.h>
#define MAX 10
int graph[MAX][MAX];
int visited[MAX];
int queue[MAX];
int front = -1, rear = -1;
int vertices;
// Queue operations for BFS
void enqueue(int value)
{
    queue[++rear] = value;
}
int dequeue()
{
    return queue[++front];
}
// BFS Traversal
void BFS(int start)
{
    int i;

    printf("BFS Traversal: ");

    visited[start] = 1;
    enqueue(start);

    while(front != rear)
    {
        int node = dequeue();

        printf("%d ", node);

        for(i = 0; i < vertices; i++)
        {
            if(graph[node][i] == 1 && visited[i] == 0)
            {
                visited[i] = 1;
                enqueue(i);
            }
        }
    }
}

// DFS Traversal
void DFS(int start)
{
    int i;

    printf("%d ", start);

    visited[start] = 1;

    for(i = 0; i < vertices; i++)
    {
        if(graph[start][i] == 1 && visited[i] == 0)
        {
            DFS(i);
        }
    }
}

int main()
{
    int edges;
    int i, src, dest;
    int start;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    // Initialize graph
    for(i = 0; i < vertices; i++)
```

```

    {
        visited[i] = 0;

        for(int j = 0; j < vertices; j++)
        {
            graph[i][j] = 0;
        }
    }

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    // Input edges
    for(i = 0; i < edges; i++)
    {
        printf("Enter edge (source destination): ");
        scanf("%d %d", &src, &dest);

        graph[src][dest] = 1;
        graph[dest][src] = 1; // Undirected graph
    }

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    // BFS
    BFS(start);

    // Reset visited array for DFS
    for(i = 0; i < vertices; i++)
    {
        visited[i] = 0;
    }

    printf("\nDFS Traversal: ");

    // DFS
    DFS(start);

    return 0;
}

```

Output:

```

Enter number of vertices: 5
Enter number of edges: 4
Enter edge (source destination): 0 1
Enter edge (source destination): 1 2
Enter edge (source destination): 2 3
Enter edge (source destination): 3 4
Enter starting vertex: 0
BFS Traversal: 0 1 2 3 4
DFS Traversal: 0 1 2 3 4
Process returned 0 (0x0)   execution time : 20.246 s
Press any key to continue.
|

```